Optimization of Edge-Native Autonomous Video Repurposing Pipelines: A Mid-2026 State-of-the-Art Analysis

The Theoretical Landscape of Edge-Based Video Repurposing

The engineering of an autonomous video repurposing agent constrained to an edge-native hardware environment requires a rigorous orchestration of computational resources, memory bandwidth, and algorithmic efficiency. In the mid-2026 artificial intelligence ecosystem, the deployment of multimodal pipelines on consumer-grade mobile Graphics Processing Units (GPUs) is no longer solely dictated by raw computational throughput, but rather by the delicate management of Video Random Access Memory (VRAM).[1] Video repurposing—a workflow that inherently involves decoding high-resolution temporal media, transcribing and aligning acoustic data, identifying semantic engagement hooks, and executing geometric spatial crops—demands the simultaneous or rapidly sequential execution of multiple deep neural networks.

The baseline architecture under evaluation targets an NVIDIA RTX 5080 Mobile GPU featuring 16 gigabytes of VRAM.[2] While this hardware provides exceptional generative capabilities bolstered by fifth-generation Tensor Cores and native FP4/FP8 precision optimizations [4], a 16GB memory ceiling strictly governs the pipeline's operational topography. To maintain fluid execution and leave adequate capacity for the operating system, display drivers, and the active high-definition video frame buffer, a hard constraint of 8GB of VRAM per active model must be enforced. Exceeding this limit forces the system to page memory across the Peripheral Component Interconnect Express (PCI-e) bus to the system's dynamic random-access memory (DRAM), a process that induces catastrophic latency spikes and destroys the near-real-time responsiveness required for autonomous video editing workflows.[1]

This report provides an exhaustive evaluation of the proposed baseline model stack, systematically comparing its components against the absolute state-of-the-art open-weights architectures available as of mid-2026. The analysis encompasses automated speech recognition (ASR) and speaker diarization alternatives, optimizations for Vision-Language Models (VLMs) to eliminate context-switching latency during viral hook identification, the unification of computer vision tracking matrices, and the strategic offloading of high-level semantic logic to the Gemini 3 Flash Application Programming Interface (API).

Topographical Constraints and the Memory Bandwidth Bottleneck

Before evaluating the individual neural network architectures, it is necessary to establish the physical constraints of the target hardware. The RTX 5080 Mobile, while a formidable edge processor, operates within strict thermal and bandwidth limitations compared to data center equivalents. In large language models and vision-language models, token generation and visual prefilling are fundamentally memory-bound operations.[1] The speed at which a model can process a video frame or generate a text token is directly tied to how rapidly the model weights can be shuttled from VRAM into the streaming multiprocessors.

When a pipeline relies on sequentially loading distinct, large-scale models—such as transitioning from an audio transcription model to a 7-billion-parameter vision-language model—the operating system incurs a massive context-switching penalty.[6] If a heavily quantized 7B model occupies 7.5GB of VRAM, swapping it out to load a computer vision tracking matrix flushes the key-value (KV) cache.[7] Upon reloading the vision-language model for the next video segment, the system must recompute the KV cache for the entire video context, resulting in severe Time-to-First-Token (TTFT) degradation.[9] Therefore, the overarching optimization strategy for this autonomous agent must prioritize architectures that remain permanently resident in memory or possess such minute parameter counts that initialization occurs in low single-digit milliseconds.

Acoustic Perception: Speech Recognition and Temporal Alignment

The baseline pipeline employs the Parakeet Token-and-Duration Transducer (TDT) 0.6B v2 via the NVIDIA NeMo framework for audio transcription, coupled sequentially with NeMo TitaNet for speaker diarization. The primary objective in video repurposing is achieving a harmonious balance between transcription throughput, Word Error Rate (WER), and absolute word-level timestamp precision, as these timestamps govern the chronological cutting logic of the final video export.

Architectural Critique of the Parakeet TDT Paradigm

The Parakeet TDT architecture represents a significant departure from standard Connectionist Temporal Classification (CTC) models. Traditional CTC models suffer from conditional independence assumptions and require extensive blank-symbol padding to align asynchronous speech frames with output text. The TDT framework circumvents this by predicting both the token and its emission duration simultaneously, allowing the decoder to skip redundant acoustic frames entirely. The 0.6B v2 iteration utilizes a Fast Conformer encoder, operating with profound computational efficiency. In benchmark testing, it often exceeds an inverse real-time factor (RTFx) of 3000 on edge processors, meaning it can transcribe hours of audio in mere

seconds.[10]

However, the TDT architecture exhibits distinct functional limitations when applied specifically to autonomous video repurposing workloads. The model fundamentally lacks custom dictionary support, rendering it highly susceptible to phonetic hallucination when encountering domain-specific vocabulary, modern internet slang, acronyms, or non-standard brand pronunciations often found in viral content.[12] More critically, while the TDT model excels in raw transcription speed, its timestamp alignment is inherently coarse compared to strictly aligned frameworks.[13] Because it predicts token durations probabilistically and skips acoustic frames, the precise millisecond boundaries of words are blurred. In video editing, where a cut must occur exactly on the plosive consonant of a spoken word to maintain a dynamic pace, coarse timestamps lead to jarring audio-visual desynchronization.

State-of-the-Art Transcription Alternatives

The mid-2026 landscape offers several highly optimized alternatives that balance conversational accuracy and timing precision against the strict 8GB VRAM constraint:

The Canary Qwen 2.5B model currently achieves the lowest Word Error Rate (5.63%) on the Hugging Face Open ASR Leaderboard.[10] This architecture pairs NVIDIA's Fast Conformer acoustic encoder with a robust Large Language Model (LLM) decoder, yielding unparalleled accuracy in complex acoustic environments.[11] However, its 2.5 billion parameters push the memory constraints dangerously close to the 8GB limit when accounting for the audio context buffer, and its RTFx of 418 represents an eight-fold reduction in throughput compared to Parakeet TDT.[10] For a mobile pipeline, Canary is computationally prohibitive.

Conversely, the Whisper ecosystem has evolved significantly to address its historical latency issues. Distil-Whisper Large-v3 provides a mathematically superior equilibrium for this specific use case.[16] Distilled directly from the OpenAI Whisper Large-v3, this variant compresses the original 1.54 billion parameters down to 756 million.[17] The knowledge distillation process operates by freezing the entire encoder and reducing the decoder to merely two initialized layers.[17] This structural pruning achieves an inference acceleration six times greater than its predecessor while maintaining a WER within one percent of the original Large-v3 on out-of-distribution audio.[17]

Crucially, the Whisper architecture can be wrapped within the WhisperX framework.[13] WhisperX is not a standalone neural network, but rather a comprehensive transcription orchestration pipeline. It utilizes Voice Activity Detection (VAD) to segment the audio, runs the transcription via Distil-Whisper, and then utilizes a wav2vec2 model to force-align the output.[13] Forced alignment maps the generated text tokens back to the raw acoustic waveform at the millisecond level, producing highly accurate word-level bounding boxes in the audio temporal space.[18] For an autonomous video agent, this wav2vec2 alignment ensures that visual framing

shifts or b-roll insertions trigger perfectly in sync with the speaker's cadence.

Diarization Pipeline Consolidation

Speaker diarization—the process of assigning speech segments to distinct, identified speakers—forms the foundation for speaker-tracking visual algorithms. The baseline stack utilizes NeMo TitaNet for this task. TitaNet extracts speaker embeddings with remarkable discriminative capacity, leveraging 1D depth-wise separable convolutions to capture acoustic features. In benchmark evaluations involving short conversational recordings, TitaNet yielded an Equal Error Rate (EER) of 1.91%, demonstrating excellent intra-class clustering and inter-class separation.[19]

However, evaluating TitaNet against Pyannote 4.0 and ECAPA-TDNN reveals critical operational dynamics that affect the overall pipeline architecture. ECAPA-TDNN architectures demonstrate slightly superior biometric performance with an EER of 1.71% and highly efficient inference latency averaging 69.43 milliseconds per frame.[19] Pyannote, which has become the de facto standard for open-source diarization, utilizes sophisticated agglomerative clustering over robust neural embeddings to identify speakers dynamically, without requiring a predefined estimation of the speaker count.[20]

The primary inefficiency in the baseline stack is the isolation of the transcription and diarization models. Running NeMo TitaNet requires an independent pass over the audio file, generating embeddings that must then be manually reconciled with the Parakeet TDT transcript timestamps. This creates a computational bottleneck and redundant memory allocation. When deploying the aforementioned WhisperX pipeline, the Pyannote diarization module is natively integrated.[18] WhisperX routes the audio through Pyannote, generates the speaker clusters, and maps the distilled text directly to those clusters via the wav2vec2 forced alignment.[22]

By migrating to the unified WhisperX pipeline powered by Distil-Whisper Large-v3 and Pyannote, the agent consolidates acoustic feature extraction. This singular process occupies roughly 3.0GB of VRAM in FP16 precision, comfortably satisfying the 8GB constraint while completely eliminating the need to pass massive audio tensors between disjointed libraries.[16]

Acoustic Model Architecture	**Parameter Count**	**VRAM Requirement (FP16)**	**Accuracy Benchmark (WER / EER)**	**Timestamp Alignment Mechanism**
Parakeet TDT 0.6B v2	600M	~1.5 GB	6.05% WER	Transducer Emission (Coarse)

Distil-Whisper Large-v3	756M	~3.0 GB	~8.00% WER	wav2vec2 Forced Alignment (Precise)
Canary Qwen 2.5B	2.5B	~6.0 GB	5.63% WER	Conformer Attention
NeMo TitaNet	N/A (Embedding)	~1.0 GB	1.91% EER	N/A
Pyannote (via WhisperX)	N/A (Clustering)	~1.2 GB	SOTA Diarization	Agglomerative Clustering

Vision-Language Cognition: Mitigating the Context-Switching Bottleneck

The most computationally intensive phase of autonomous video repurposing is visual scene understanding. The agent must scan incoming video frames to identify zero-shot viral hooks—moments of high emotion, sudden action, compelling visual transitions, or relevant optical character recognition (OCR) data. The baseline implementation attempts to achieve this using the Qwen2.5-VL-7B-Instruct model, heavily quantized to an FP8 format to squeeze into the memory budget.

The Latency Cost of 7-Billion Parameter Models on Mobile Edge

While the Qwen2.5-VL-7B architecture is undeniably capable, featuring Multimodal Rotary Position Embeddings (M-RoPE) and dynamic resolution handling [23], deploying a 7-billion parameter model on a 16GB mobile GPU introduces fatal systemic bottlenecks. An FP8 quantized 7B model occupies approximately 7.5GB of VRAM at rest. However, this figure represents only the static model weights. During inference, the model must construct a Key-Value (KV) cache for the provided context.

When processing temporal media, the context window expands exponentially. Analyzing just 30 seconds of video requires the model to encode dozens of sequential frames. Even with state-of-the-art architectures, a 7B multimodal model processing a 30-second video snippet can demand upwards of 41.85GB of VRAM in unquantized states, and roughly 17.84GB even

under extreme 4-bit Advanced Weight Quantization (AWQ).[25] Consequently, relying on a 7B parameter model for continuous video analysis on a 16GB GPU guarantees immediate memory saturation.

When the VRAM fills, the operating system initiates memory paging, offloading active tensors to the CPU's much slower DRAM. In benchmark testing utilizing frameworks like vLLM and Ollama, this phenomenon manifests as extreme Time-to-First-Token (TTFT) delays.[6] The CPU utilization spikes to 100% as it attempts to manage the overflow, while the GPU utilization plummets because it is starved of data waiting for the PCI-e bus to transfer the necessary weights.[9] For a video editing agent that must make sub-second decisions regarding scene retention, latency spikes of several seconds per frame render the pipeline unworkable.

Sub-7GB High-Efficiency Vision-Language Architectures

To eliminate memory paging and context-switching latency, the pipeline must abandon the 7B paradigm and adopt significantly lighter architectures that maintain zero-shot visual comprehension capabilities while strictly adhering to a sub-4GB VRAM footprint. The mid-2026 open-weights ecosystem provides several highly optimized multimodal alternatives tailored for edge deployment.

SmolVLM (2-Billion Parameters): Developed specifically to minimize memory footprints for edge devices, SmolVLM leverages the SmolLM2 1.7B causal language backbone coupled with a shape-optimized SigLIP vision encoder.[26] Its critical innovation lies in its highly aggressive visual compression algorithm. Unlike larger models that pass massive arrays of visual tokens to the language decoder, SmolVLM applies a pixel shuffle strategy that compresses patched visual data by a factor of nine.[26] It processes 384x384 pixel image patches into a mere 81 tokens.[26]

For video processing tasks, SmolVLM extracts up to 50 evenly sampled frames from the sequence, avoiding internal resizing to maintain granular detail while keeping the token count astonishingly low.[26] This extreme token efficiency allows the model to operate with a baseline VRAM requirement of just 5.02GB. In comparative prefill throughput benchmarks, SmolVLM outpaces Qwen2-VL by factors of 3.3 to 4.5, effectively eliminating the TTFT latency spikes.[26] While exceptional for static frame analysis, SmolVLM exhibits documented limitations in complex temporal understanding, occasionally struggling to discern whether rapid actions occur simultaneously or sequentially across frames.[26]

Qwen2.5-Omni-3B: Designed natively for multimodal omni-perception, the 3B variant of the Qwen2.5-Omni series represents a significant leap in holistic media understanding.[25] The architecture utilizes Time-aligned Multimodal Rotary Position Embeddings (TMRoPE) to synchronize temporal video inputs meticulously with accompanying audio tracks.[25] This allows the model to analyze the semantic relationship between a visual action and a sound effect natively. When quantized using AWQ or GPTQ-Int4 protocols, the model's memory consumption is dramatically reduced.[25] However, processing a full 60-second video segment

still necessitates approximately 10GB of VRAM due to the sheer volume of temporal frames.[28] Thus, while significantly smaller than the 7B variant, the Omni-3B requires aggressive frame subsampling algorithms to remain viable under the 8GB ceiling.

Qwen3-4B and Qwen3-Omni-30B-A3B (Mixture of Experts): The third generation of Qwen architectures introduces paradigm-shifting methodologies in cognitive processing. The Qwen3-4B model features a unified architecture with dynamic dual-mode operation, allowing it to toggle autonomously between a "thinking" state for deep logical evaluation and a "non-thinking" state for rapid, low-latency execution.[29] Built on 36 transformer layers with Grouped Query Attention (GQA), the 4B model ensures high computational throughput and occupies roughly 3.5GB of VRAM under INT4 quantization.[31] It accommodates an enormous native context window of up to 262,144 tokens, making it ideal for ingesting long strings of video frames.[31]

Even more profound is the Qwen3-Omni-30B-A3B. This model utilizes a Mixture of Experts (MoE) paradigm.[30] While the total parameter count is 30 billion, the routing algorithm only activates approximately 3 billion parameters per forward pass.[30] This allows the model to deliver the analytical depth and causal reasoning of a massive data-center architecture while executing with the VRAM footprint and speed of a 3B model.[34] Users have reported VRAM usage as low as 2.6GB for the active experts in highly optimized deployments.[34]

Vision-Language Model Architecture	**Active Parameters**	**Quantized VRAM Baseline (INT4/FP8)**	**Video Token Compression Strategy**	**Prefill Latency Profile**
Qwen2.5-VL-7B-Instruct	7B	~7.5 GB	Standard Dynamic Resolution	High (Bandwidth Bound)
SmolVLM	2B	~2.5 GB	9x Pixel Shuffle	Extremely Low
Qwen2.5-Omni-3B	3B	~3.5 GB	TMRoPE Sequential	Moderate

Qwen3-4B	4B	~3.8 GB	GQA / Dual-Mode Thinking	Low
Qwen3-Omni-30B-A3B (MoE)	3B (Per Pass)	~4.5 GB	Mixture of Experts Routing	Very Low

For the specific task of zero-shot viral hook identification, where the model must interpret visual sentiment, optical character recognition (OCR) elements, and action sequences rapidly, the Qwen3-4B model presents the optimal frontier. Its dual-mode functionality allows the autonomous agent to engage rapid, non-thinking execution for standard frame scanning, bypassing deep evaluation layers to save milliseconds.[29] When a potential viral hook is detected—such as a sudden change in scene luminance, a high-contrast text overlay, or an exaggerated human expression—the model can dynamically activate its thinking mode for intensive logical evaluation of the frame's engagement potential.[29] This dynamic scaling ensures VRAM is never saturated unnecessarily, completely mitigating the context-switching latency that plagues the FP8 Qwen2.5-VL-7B in the baseline stack.

Spatial Dynamics: Unifying Computer Vision Tracking Matrices

Once a viral hook is identified and the audio is perfectly aligned, the autonomous agent must visually track the subject to execute dynamic crops (e.g., converting a 16:9 landscape video into a 9:16 portrait format for social media). The baseline architecture relies on a bifurcated computer vision approach: YOLOv12-Face Nano for facial bounding box detection, followed by YOLO26n-Pose coupled with the ByteTrack algorithm for anatomical keypoint estimation and temporal tracking.

Running dual object detection models sequentially across every frame of a video compounds inference latency, doubles the PyTorch execution overhead, and fragments the VRAM allocation unnecessarily.

Critique of the Dual-Model Paradigm and Attention Mechanisms

YOLOv12 introduced a radical architectural shift, moving away from conventional Convolutional Neural Networks (CNNs) toward an attention-centric design heavily reliant on Area Attention Mechanisms and Residual Efficient Layer Aggregation Networks (R-ELAN).[35] While this departure yielded exceptionally high precision for complex spatial tasks like facial recognition

under occlusion, it introduced significant computational penalties.

Self-attention mechanisms inherently demand elevated memory consumption as they must compute the relationships between all spatial patches in an image.[35] Furthermore, these architectures exhibit notably slower processing throughput on edge-bound Central Processing Units (CPUs) and mobile GPUs compared to optimized convolutions.[35] Deploying YOLOv12-Face on an RTX 5080 Mobile squanders critical memory bandwidth and compute cycles that should be strictly reserved for the video frame buffer and the VLM.

Furthermore, passing cropped matrices from a facial detector (YOLOv12) into a separate pose estimator (YOLO26n-Pose) creates a synchronous programming bottleneck. If the facial detection matrix stalls or drops a frame due to motion blur, the pose estimation tensor is starved of input, preventing continuous pipeline execution and causing the ByteTrack algorithm to lose the subject's tracking ID.

The YOLO26 End-to-End Unified Architecture

Released in 2025 as the definitive edge-first evolution of the YOLO lineage, YOLO26 fundamentally restructures the detection pipeline.[37] The architecture's most significant technological leap is the complete eradication of Non-Maximum Suppression (NMS) as a post-processing step.[39] Traditional object detectors generate thousands of overlapping bounding box predictions, relying on NMS algorithms running on the CPU to filter out redundant boxes based on intersection-over-union thresholds. This creates a severe CPU bottleneck.

By employing a native end-to-end predictor, YOLO26 outputs a single, refined detection for each object directly from the neural network head, entirely eliminating post-processing latency.[38] Additionally, the removal of the Distribution Focal Loss (DFL) module significantly streamlines the export graph, ensuring maximum compatibility with accelerated edge inference engines like NVIDIA TensorRT.[39]

Crucially for the video repurposing agent, YOLO26 is a consolidated multi-task framework. The architecture supports object detection, instance segmentation, oriented bounding boxes, and pose estimation simultaneously through shared backbone features.[37] The pose estimation variant, YOLO26-Pose, is trained extensively on the COCO keypoints dataset.[40] It natively extracts 17 anatomical keypoints representing the human skeleton. Most importantly for this use case, these 17 points include five specific facial landmarks: the nose, left eye, right eye, left ear, and right ear.[40]

Single-Pass Pose and Bounding Extraction

By migrating the computer vision stack exclusively to YOLO26-Pose (specifically the small or medium variant, such as YOLO26s-Pose, which occupies less than 150MB of VRAM), the pipeline can extract both the spatial bounding coordinates of the subject and the precise facial

landmarks in a single forward pass.[39]

The mathematical advantage lies in the shared convolution layers and the novel training regime. The YOLO26 architecture utilizes the MuSGD optimizer—a hybrid of Stochastic Gradient Descent and the Muon optimizer—which ensures the model converges rapidly across complex, multi-objective loss landscapes.[39] When an image frame is ingested, the network simultaneously optimizes the bounding box regression loss and the pose Residual Log-Likelihood Estimation (RLE) loss.[39]

Consequently, the output tensor natively yields the x, y coordinates of the full-body bounding box (which is fed directly into the ByteTrack algorithm to maintain temporal identity across frames) and the spatial geometry of the facial features (which is fed into the framing algorithm to ensure the subject's face remains centered in the vertical 9:16 crop).[40]

This unification completely eradicates the need for YOLOv12-Face Nano. Dropping the attention-heavy YOLOv12 frees approximately 300MB to 500MB of VRAM, reduces the pipeline inference time by half, and massively simplifies the data flow into the ByteTrack association algorithm, ensuring rock-solid subject tracking for automated reframing.

Computer Vision Model	**Primary Architecture**	**NMS Requirement**	**Extracted Features**	**Inference Paradigm**
YOLOv12-Face Nano	Area Attention / R-ELAN	Required (CPU Bound)	Facial Bounding Box	Sequential Pass 1
YOLO26n-Pose	CNN / MuSGD	End-to-End (NMS-Free)	17-Point Skeletal Matrix	Sequential Pass 2
YOLO26s-Pose (Unified)	CNN / MuSGD	End-to-End (NMS-Free)	Bounding Box + 5 Facial Landmarks	**Single Simultaneous Pass**

Cloud-Edge Synergy: Cognitive Offloading to Gemini 3 Flash

The autonomous agent operates in a hybrid topology, leveraging local compute for high-frequency operations—such as video frame tracking, acoustic transcription, and rapid hook identification—and relying on the Gemini 3 Flash API for high-level scripting, metadata extraction, and complex semantic evaluation. Maximizing the efficiency of the RTX 5080 Mobile demands the ruthless offloading of overlapping cognitive tasks to the cloud infrastructure.[41]

Gemini 3 Flash, released as Google's production-ready execution layer, introduces several advanced control parameters that directly dictate the latency-cost frontier of remote execution, most notably the thinking_level and media_resolution constraints.[43]

Modulating Execution Latency via Thinking Levels

Gemini 3 Flash incorporates an adjustable cognitive depth parameter. The thinking_level can be modulated between MINIMAL, LOW, MEDIUM, and HIGH.[43] In previous iterations of large language models, invoking complex script analysis or semantic evaluation required full-depth processing, resulting in high latency as the model generated extensive hidden chain-of-thought tokens before producing an answer.

For a video repurposing agent generating metadata—such as synthesizing viral titles from transcripts, extracting SEO keywords, or summarizing the narrative arc of a clip—the MINIMAL or LOW configuration is mathematically optimal.[45] Setting thinking_level=MINIMAL forces the model to bypass deep multi-step planning loops, acting strictly as a high-throughput, low-latency execution layer.[46]

This architecture permits the local system to stream the perfectly aligned text generated by the local Distil-Whisper model directly to the Gemini API, receiving fully formatted JSON metadata in a matter of milliseconds. Offloading all text-based semantic synthesis entirely to Gemini 3 Flash under the MINIMAL setting ensures the local RTX 5080 devotes zero GPU cycles to natural language generation tasks, reserving all available compute for processing incoming video frames.[46]

Deprecating Local Image Cropping via Agentic Vision

The most profound paradigm shift in the Gemini 3 Flash architecture is the introduction of "Agentic Vision".[47] Historically, Vision-Language Models processed images in a static, single-pass glance. If a detail was small, obscured, or required mathematical counting, the model was forced to rely on statistical hallucination rather than deterministic observation.[47] To combat this, local pipelines often employed complex iterative logic: utilizing local YOLO models to detect regions of interest, algorithmically cropping those regions using libraries like OpenCV, and feeding the high-resolution crops back into the VLM.

Gemini 3 Flash natively internalizes this entire process through an autonomous Think-Act-Observe loop powered by secure, backend Python code execution.[47] When supplied

with a high-definition video frame and a complex query, the model operates iteratively:

1. **Think**: The system analyzes the user query and the initial wide-angle image, formulating a spatial investigation plan (e.g., "The subject's expression is too small to read; I must zoom in on the facial coordinates").[47]
2. **Act**: The model writes and executes Python code internally to calculate pixel coordinates, draw bounding boxes, or execute geometric crops on the uploaded image tensor.[47]
3. **Observe**: The freshly cropped or mathematically annotated tensor is appended back into the model's context window, allowing it to inspect the new, high-resolution data before generating a final semantic response.[47]

This capability renders local deterministic cropping algorithms obsolete for high-level semantic tasks. Instead of burdening the RTX 5080 with calculating spatial attention crops using YOLO just to feed the local Qwen model for expression analysis, the raw, uncropped frame can be passed directly to Gemini 3 Flash via the API.[47]

By setting the media_resolution parameter to HIGH (which allocates up to 1,120 tokens per image to preserve fine-grained pixels) [43], Gemini's Agentic Vision will autonomously zoom, isolate subjects, and evaluate the scene. Benchmark data indicates that enabling code execution yields a consistent 5% to 10% quality boost across complex vision tasks compared to static evaluation.[47] Consequently, any task requiring dense spatial reasoning, OCR extraction from background signage, or complex scene comprehension should be stripped from the local Qwen3-4B workload and routed to Gemini 3 Flash. The local VLM should be restricted strictly to identifying broad temporal transitions and basic semantic hooks to trigger the API calls.

Final Pipeline Synthesis and VRAM Allocation Topology

The exhaustive evaluation of the mid-2026 machine learning landscape against the strict confines of an RTX 5080 Mobile (16GB VRAM) necessitates a complete structural revision of the proposed baseline stack. By eliminating architectural redundancies, enforcing unified tensor operations, and aggressively exploiting the deterministic capabilities of the Gemini 3 Flash API, the pipeline achieves an optimal state of latency, precision, and memory availability.

The Optimized Local Stack Architecture

1. **Acoustic Perception and Temporal Alignment:**
   - *Baseline*: Parakeet TDT 0.6B v2 + NeMo TitaNet.
   - *Optimized Deployment*: **WhisperX featuring Distil-Whisper Large-v3 and Pyannote 4.0**.
   - *Architectural Rationale*: This configuration merges acoustic feature extraction. Distil-Whisper provides six times the speed of standard large models while strictly adhering to wav2vec2 forced-alignment timestamps for frame-accurate video cuts.

Pyannote seamlessly clusters these precise timestamps into discrete speakers via agglomerative clustering without relying on a separate TitaNet pass, ensuring perfect audio-visual synchronization while occupying only ~3.0GB of VRAM in FP16.

2. **Vision-Language Cognition (Viral Hook Identification):**
   - *Baseline*: Qwen2.5-VL-7B-Instruct (FP8).
   - *Optimized Deployment*: **Qwen3-4B (INT4 Quantization)**.
   - *Architectural Rationale*: The 7B FP8 model saturated the PCI-e bus during context switching, destroying Time-to-First-Token metrics as it repeatedly built the KV cache. The Qwen3-4B model operates securely within a 3.8GB VRAM footprint. Its dual-mode functionality ensures it executes rapidly without invoking deep logical layers unless specifically triggered. This completely prevents memory paging while maintaining exceptional zero-shot hook detection across a massive 262k context window.
3. **Spatial Geometry and Temporal Tracking:**
   - *Baseline*: YOLOv12-Face Nano + YOLO26n-Pose.
   - *Optimized Deployment*: **YOLO26s-Pose (Single Pass)**.
   - *Architectural Rationale*: Running an attention-heavy YOLOv12 model sequentially with a pose model fractures the execution graph and bottlenecks the CPU. YOLO26 utilizes an NMS-free, end-to-end architecture with MuSGD optimization. A single inference pass on YOLO26s-Pose natively yields both the human bounding box for ByteTrack association and the 17-point anatomical matrix (including all 5 facial landmarks) necessary for dynamic portrait framing. This reduces the computer vision VRAM footprint to under 200MB and halves pipeline inference latency.
4. **Cloud-Edge Load Balancing (Gemini 3 Flash Synergy):**
   - *Strategic Implementation*: All post-processing metadata generation, script synthesis, and complex visual geometry analysis are systematically offloaded to Gemini 3 Flash.
   - *Parameter Tuning*: Metadata text generation relies on the thinking_level=MINIMAL parameter to ensure ultra-low latency execution without excess logic loops. Deep scene analysis relies on media_resolution=HIGH paired with Agentic Vision, allowing Google's infrastructure to execute Python-based zooming and cropping autonomously, completely relieving the local RTX 5080 of iterative spatial processing.

Operational VRAM Memory Map

Under the optimized framework, the RTX 5080 Mobile's 16GB memory map operates with profound efficiency, completely averting the risk of bus saturation:

- **Operating System & Display Driver Overhead**: ~2.0 GB
- **Video Frame Buffer & Active Tracking Tensors**: ~1.5 GB
- **Computer Vision Matrix (YOLO26s-Pose)**: ~0.2 GB
- **Acoustic Subsystem (WhisperX / Pyannote)**: ~3.0 GB
- **Vision-Language Model (Qwen3-4B INT4)**: ~3.8 GB
- **Available Headroom for Burst Execution**: **~5.5 GB**

This topology deliberately preserves over 5GB of unallocated VRAM. In an asynchronous autonomous agent, memory spikes occur unpredictably when large frame batches are decoded or when audio matrices temporarily overlap during complex scene transitions. The generous memory headroom eliminates the risk of system paging to the CPU, ensuring the pipeline remains purely GPU-bound. By executing rapid temporal evaluations locally through unified models and deferring dense semantic synthesis to the highly tunable Gemini 3 Flash API, the architecture achieves a continuous, stall-free workflow fully capable of real-time autonomous video repurposing at the edge.

Works cited

1. The RTX 5080 Is 3x Faster Than NVIDIA's $3999 “AI Supercomputer” — And It Doesn't Matter | by Mubashir Rahim - Medium, accessed March 7, 2026, https://medium.com/@mubashir_rahim/the-rtx-5080-is-3x-faster-than-nvidias-3-999-ai-supercomputer-and-it-doesn-t-matter-fbd0fb575ed7
2. RTX 50 Gaming Laptops 2026: Complete Performance Guide - Newegg, accessed March 7, 2026, https://www.newegg.com/insider/gaming-laptops-in-2026-the-rtx-50-series-revolution/
3. Benchmarking AI on an RTX 5080: How Well Do Popular LLMs Run? - Micro Center, accessed March 7, 2026, https://www.microcenter.com/site/mc-news/article/benchmarking-ai-on-nvidia-5080.aspx
4. GeForce RTX 5080 Graphics Cards - NVIDIA, accessed March 7, 2026, https://www.nvidia.com/en-us/geforce/graphics-cards/50-series/rtx-5080/
5. NVIDIA RTX Accelerates 4K AI Video Generation on PC With LTX-2 and ComfyUI Upgrades, accessed March 7, 2026, https://blogs.nvidia.com/blog/rtx-ai-garage-ces-2026-open-models-video-generation/
6. Ollama vs. vLLM: A deep dive into performance benchmarking | Red Hat Developer, accessed March 7, 2026, https://developers.redhat.com/articles/2025/08/08/ollama-vs-vllm-deep-dive-performance-benchmarking
7. vLLM vs Ollama: Key differences, performance, and how to run them | Blog - Northflank, accessed March 7, 2026, https://northflank.com/blog/vllm-vs-ollama-and-how-to-run-them
8. Ollama VRAM Requirements: Complete 2026 Guide to GPU Memory for Local LLMs, accessed March 7, 2026, https://localllm.in/blog/ollama-vram-requirements-for-local-llms
9. [Performance]: Multi-Modal Benchmark on NVIDIA A100 – Qwen2.5-VL / MiniCPM-V-4 / InternVL3_5-4B / InternVL3_5-2B · Issue #24728 · vllm-project/vllm - GitHub, accessed March 7, 2026, https://github.com/vllm-project/vllm/issues/24728
10. The Top Open Source Speech-to-Text (STT) Models in 2025 - Modal, accessed

March 7, 2026, https://modal.com/blog/open-source-stt
11. Open ASR Leaderboard: Trends and Insights with New Multilingual & Long-Form Tracks - Hugging Face, accessed March 7, 2026, https://huggingface.co/blog/open-asr-leaderboard
12. 30 Days Testing Parakeet v3 vs Whisper : r/LocalLLaMA - Reddit, accessed March 7, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1nf10ye/30_days_testing_parakeet_v3_vs_whisper/
13. Choosing between Whisper variants: faster-whisper, insanely-fast-whisper, WhisperX, accessed March 7, 2026, https://modal.com/blog/choosing-whisper-variants
14. TheWhisper-Turbo: SOTA Multilingual Transcription - TheStage AI, accessed March 7, 2026, https://app.thestage.ai/blog/TheWhisper-tech-report?id=7
15. What's the best local ASR model for real-time dictation in 2026? Is ..., accessed March 7, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1rm36ew/whats_the_best_local_asr_model_for_realtime/
16. Faster Whisper transcription with CTranslate2 - GitHub, accessed March 7, 2026, https://github.com/SYSTRAN/faster-whisper
17. Best open source speech-to-text (STT) model in 2026 (with benchmarks) | Blog - Northflank, accessed March 7, 2026, https://northflank.com/blog/best-open-source-speech-to-text-stt-model-in-2026-benchmarks
18. WhisperX vs Competitors: AI Transcription Comparison 2026 - BrassTranscripts, accessed March 7, 2026, https://brasstranscripts.com/blog/whisperx-vs-competitors-accuracy-benchmark
19. Comparison of Modern Deep Learning Models for Speaker Verification - MDPI, accessed March 7, 2026, https://www.mdpi.com/2076-3417/14/4/1329
20. Speaker Diarization DER performance comparison pyannoteAI Benchmark, accessed March 7, 2026, https://www.pyannote.ai/benchmark
21. Comparing state-of-the-art speaker diarization frameworks : Pyannote vs Nemo, accessed March 7, 2026, https://lajavaness.medium.com/comparing-state-of-the-art-speaker-diarization-frameworks-pyannote-vs-nemo-31a191c6300
22. Looking for diarization model better than Pyannote : r/LocalLLaMA - Reddit, accessed March 7, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1m48v53/looking_for_diarization_model_better_than_pyannote/
23. Ultimate Guide - The Best LLMs for Low-VRAM GPUs in 2026 - SiliconFlow, accessed March 7, 2026, https://www.siliconflow.com/articles/en/best-LLMs-for-low-VRAM-GPUs
24. Run Qwen2.5-VL 7B Locally: Vision AI Made Easy - Labellerr, accessed March 7, 2026, https://www.labellerr.com/blog/run-qwen2-5-vl-locally/
25. QwenLM/Qwen2.5-Omni - GitHub, accessed March 7, 2026, https://github.com/QwenLM/Qwen2.5-Omni

26. SmolVLM - small yet mighty Vision Language Model - Hugging Face, accessed March 7, 2026, https://huggingface.co/blog/smolvlm
27. Qwen/Qwen2.5-Omni-3B - Hugging Face, accessed March 7, 2026, https://huggingface.co/Qwen/Qwen2.5-Omni-3B
28. Qwen2.5-3B: Specifications and GPU VRAM Requirements - ApX Machine Learning, accessed March 7, 2026, https://apxml.com/models/qwen2-5-3b
29. Qwen3-4B: Specifications and GPU VRAM Requirements - ApX Machine Learning, accessed March 7, 2026, https://apxml.com/models/qwen3-4b
30. Qwen3: Think Deeper, Act Faster | Qwen, accessed March 7, 2026, https://qwenlm.github.io/blog/qwen3/
31. Qwen3.5-4B: Specifications and GPU VRAM Requirements - ApX Machine Learning, accessed March 7, 2026, https://apxml.com/models/qwen35-4b
32. VRAM requirements for all Qwen3 models (0.6B–32B) – what fits on your GPU? - Reddit, accessed March 7, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1kfvba4/vram_requirements_for_all_qwen3_models_06b32b/
33. Qwen3-Omni Technical Report - arXiv.org, accessed March 7, 2026, https://arxiv.org/html/2509.17765v1
34. Qwen 3 Performance: Quick Benchmarks Across Different Setups : r/LocalLLaMA - Reddit, accessed March 7, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1kdsp4z/qwen_3_performance_quick_benchmarks_across/
35. YOLO12: Attention-Centric Object Detection - Ultralytics YOLO Docs, accessed March 7, 2026, https://docs.ultralytics.com/models/yolo12/
36. YOLO Model Comparison: v8, v11, v12 & v26 | Data-Driven Engineering - APMonitor, accessed March 7, 2026, https://apmonitor.com/dde/index.php/Main/PoseEstimation
37. YOLO26: Key Architectural Enhancements and Performance Benchmarking for Real-Time Object Detection - arXiv.org, accessed March 7, 2026, https://arxiv.org/html/2509.25164v4
38. YOLO26: The Edge-First Evolution of Real-Time Object Detection | Datature Blog, accessed March 7, 2026, https://datature.io/blog/yolo26-the-edge-first-evolution-of-real-time-object-detection
39. Ultralytics YOLO26, accessed March 7, 2026, https://docs.ultralytics.com/models/yolo26/
40. Pose Estimation - Ultralytics YOLO Docs, accessed March 7, 2026, https://docs.ultralytics.com/tasks/pose/
41. Gemini 3 Flash for Enterprises | Google Cloud Blog, accessed March 7, 2026, https://cloud.google.com/blog/products/ai-machine-learning/gemini-3-flash-for-enterprises
42. Google Launches Gemini 3 Flash to Power Real-Time AI Agents at Scale, accessed March 7, 2026, https://www.mitsloanme.com/article/google-launches-gemini-3-flash-to-power-real-time-ai-agents-at-scale/

43. Gemini 3 Flash | Generative AI on Vertex AI | Google Cloud ..., accessed March 7, 2026, https://docs.cloud.google.com/vertex-ai/generative-ai/docs/models/gemini/3-flash
44. Gemini 3 Flash: Google's 3x Faster AI at 1/4 the Cost, accessed March 7, 2026, https://www.digitalapplied.com/blog/gemini-3-flash-google-ai-guide
45. Gemini 3 Developer Guide | Gemini API - Google AI for Developers, accessed March 7, 2026, https://ai.google.dev/gemini-api/docs/gemini-3
46. Get started with Gemini 3 | Generative AI on Vertex AI - Google Cloud Documentation, accessed March 7, 2026, https://docs.cloud.google.com/vertex-ai/generative-ai/docs/start/get-started-with-gemini-3
47. Introducing Agentic Vision in Gemini 3 Flash - The Keyword, accessed March 7, 2026, https://blog.google/innovation-and-ai/technology/developers-tools/agentic-vision-gemini-3-flash/
48. Beyond Just Looking: Gemini 3 Now Has Agentic Vision | by #TheGenAIGirl — code, community, and GenAI. | Google Cloud - Medium, accessed March 7, 2026, https://medium.com/google-cloud/beyond-just-looking-gemini-3-now-has-agentic-vision-b20c7c6809ce